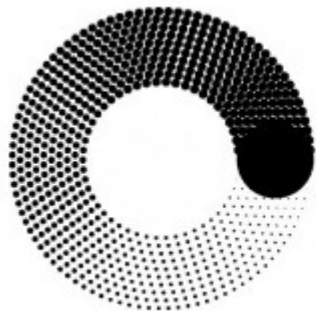




**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 4.
Непрерывная интеграция**

**Москва
2024**

Теоретические основы

Непрерывная интеграция (Continuous Integration, CI) — это практика автоматизации интеграции изменений кода от нескольких участников в один программный проект. Это основная передовая практика непрерывной разработки, позволяющая разработчикам часто объединять изменения кода в центральный репозиторий, где затем выполняются сборки и тесты. Автоматизированные инструменты используются для проверки правильности нового кода перед интеграцией.

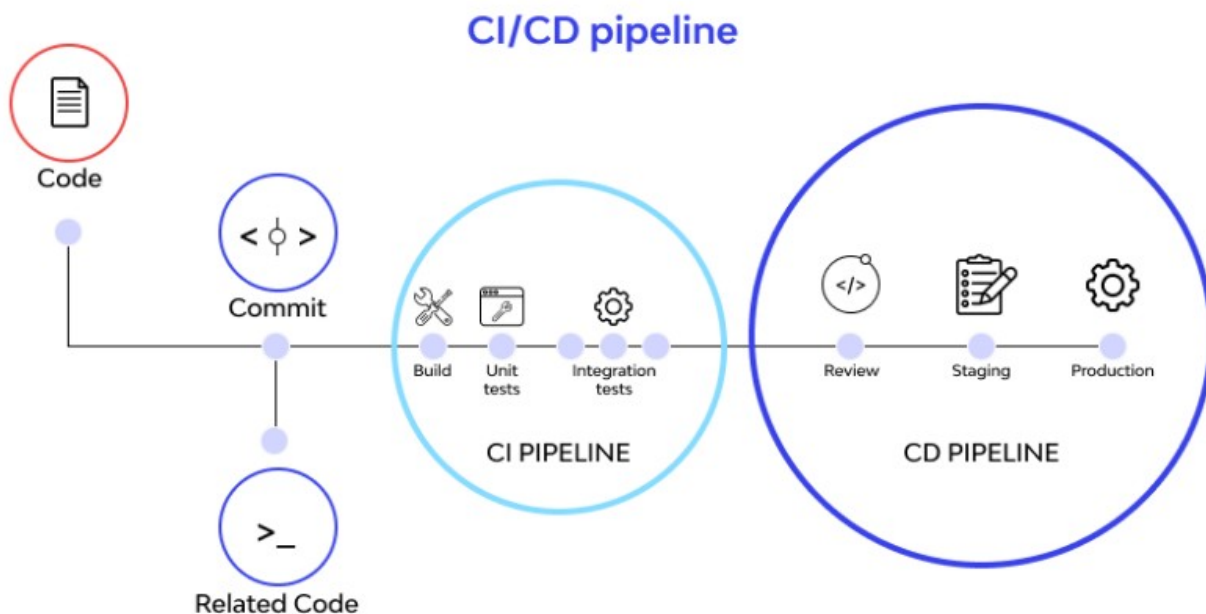
Чтобы понять важность CI, полезно сначала обсудить некоторые болевые точки, которые часто возникают из-за отсутствия CI. Без CI разработчики должны вручную координировать и сообщать, когда они вносят свой код в конечный продукт. Эта координация распространяется не только на команды разработчиков, но и на остальную часть организации. Продуктовые группы должны координировать, когда последовательно запускать функции и исправления, а также какие члены команды будут нести за это ответственность.

Накладные расходы на связь в среде без CI могут превратиться в сложную и запутанную рутинную синхронизацию, которая увеличивает ненужные бюрократические затраты на проекты. Это приводит к более медленным выпускам кода с более высоким уровнем отказов, поскольку требует от разработчиков деликатности и внимательности к интеграции. Эти риски растут в геометрической прогрессии по мере увеличения размеров команды разработчиков и кодовой базы.

Без надежного конвейера непрерывной интеграции может образоваться разрыв между инженерной командой и остальной частью организации. Связь между продуктом и инженерами может быть сложной. Инженерия становится черным ящиком, в который остальная часть команды вводит требования и функции и, возможно, получает ожидаемые результаты. Инженерам будет сложнее оценивать время доставки по запросам, потому что время для интеграции новых изменений становится неизвестным риском.

Непрерывная интеграция помогает увеличить численность персонала и производительность инженерных групп. Внедрение CI в вышеупомянутый сценарий позволяет разработчикам программного обеспечения параллельно работать над функциями независимо друг от друга. Когда они будут готовы объединить эти функции в конечный продукт, они смогут сделать это независимо и быстро. CI — ценная и хорошо зарекомендовавшая себя практика в современных высокопроизводительных организациях по разработке программного обеспечения.

Схематически процесс непрерывной интеграции (и непрерывной доставки (Continuous delivery, CD), которая будет рассмотрена позже), можно отобразить следующим образом:



Задача инструментов CI/CD заключается в координации действий между целым рядом систем, включая системы контроля версий (например, Git, Mercurial или Perforce), инструменты сборки (например, Ant, Maven или Gradle), автоматизированные среды тестирования (например, NUnit или JUnit), менеджеры пакетов (например, NuGet), средства отслеживания проблем (например, Jira, YouTrack или Github) и контейнерные платформы (например, Docker).

Когда дело доходит до построения конвейера CI/CD, универсального подхода не существует, поэтому очень важно иметь возможность интеграции с различными инструментами, будь то через встроенную поддержку, сторонние плагины или API, которые позволяют расширить функциональность в соответствии с вашими потребностями. Наличие такой гибкости упрощает расширение вашей платформы CI/CD для других команд в вашей организации в будущем.

Задания

1. При помощи команды `docker push` собранный в ходе ЛР3 образ разрабатываемого приложения загрузить в DockerHub
2. Загрузить с Docker Hub образ `jetbrains/teamcity-server` и развернуть его согласно инструкции на странице образа
3. Открыть веб-интерфейс развернутого приложения, завести пользователя-администратора
4. В панели администрирования завести по одному пользователю для каждого члена команды
5. Завести репозиторий в TeamCity, привязав репозиторий проекта
6. По выбору студента - либо из предложенных вариантов сборки, либо из реализованных студентом создать конфигурацию (последовательность шагов)

для сборки разрабатываемого приложения (для всех веток) в Docker образ и отправки его в Docker Hub (для ветки «в разработке» (dev) и ветки «в эксплуатации» (prod))

7. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).